目录

- 目录
- Z-stack
  - 前置概念说明:
  - 前提说明:
- 基本概念
  - 协调器、路由和终端
- `OSAL`
  - 执行流程
  - 所有函数说明
  - 重点关注函数
    - `osal_init_system();`
    - `osal_start_system();`
- `GenericApp` 示例的应用解析
- 小组的后续目标

大致框架,待完善...

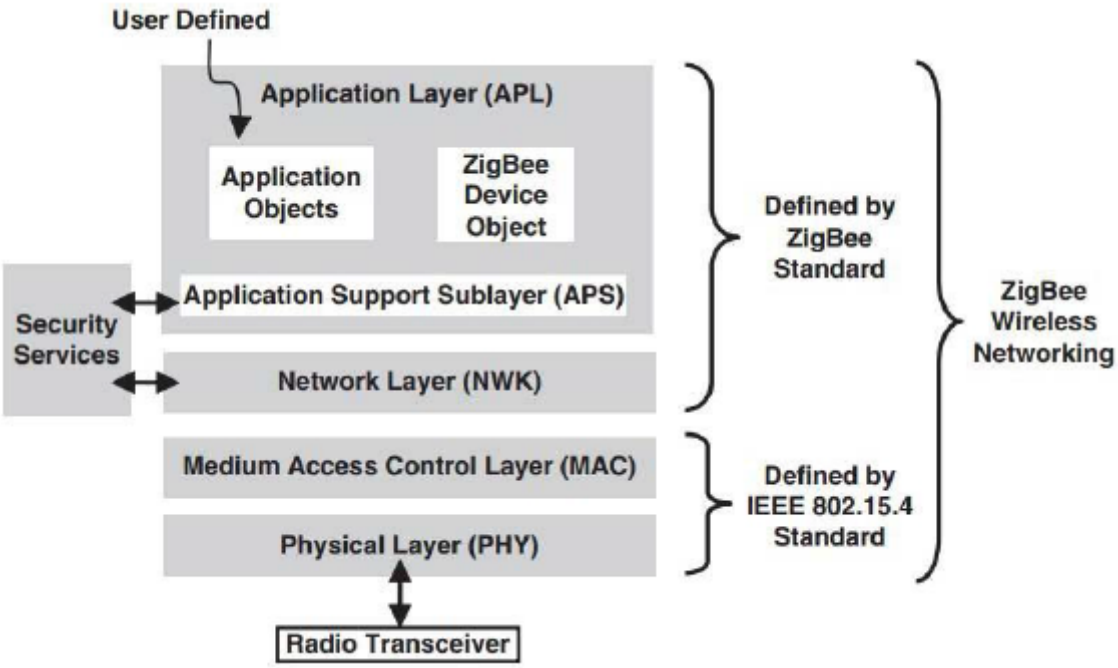
Z-stack

前置概念说明:

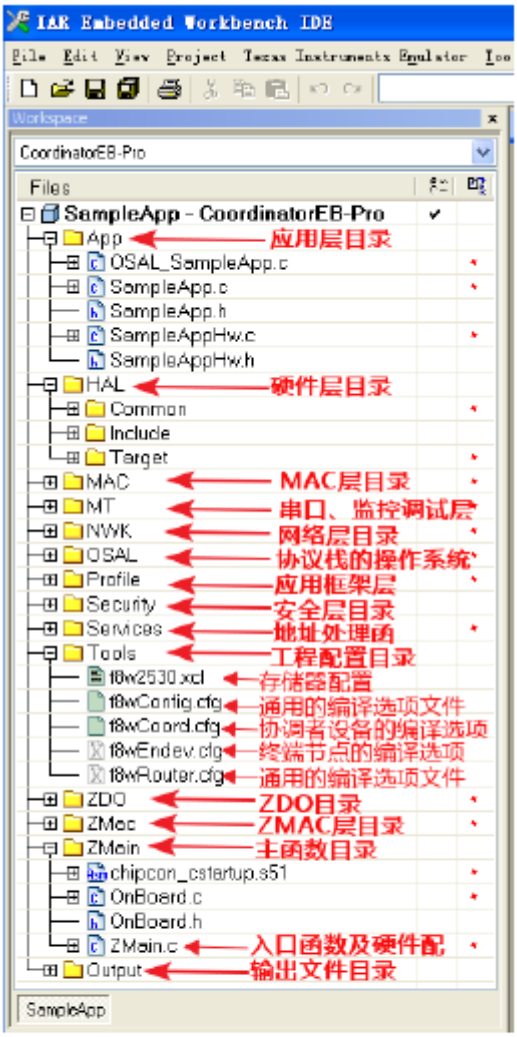
`ZigBee` ：是一种低速短距离传输的无线网上协议，是一项新型的无线通信技术

> `ZigBee`的协议分为两部分，IEEE 802.15.4 定义了 `PHY`（物理层）和 `MAC`（介质访问层）技术规范`ZigBee` 联盟定义了 `NWK`（网络层）、`APS`（应用程序支持子层）、`APL`（应用层）技术规范

协议栈 就是将各个层定义的协议都集合在一起，以函数的形式实现，并供给用户提供 `API`(应用层)，用户可以直接调用



`Z-Stack` 协议栈的各层组件声明



App：应用层目录，这是用户创建各种不同工程的区域，在这个目录中包含了应用层的内容和这个项目的主要内容。
HAL：硬件层目录，包含有与硬件相关的配置和驱动及操作函数。
MAC：MAC层目录，包含了MAC层的参数配置文件及其MAC的LIB库的函数接口文件。
MT：实现通过串口可控制各层，并与各层进行直接交互
NWK：网络层目录，包含网络层配置参数文件网络层库的函数接口文件及APS层库的函数接口。
OSAL：协议栈的操作系统。
Profile：Application framework应用框架层目录，包含AF层处理函数文件。应用框架层是应用程序和APS层的无线数据接口。
Security：安全层目录，包含安全层处理函数，比如加密函数等
Services：地址处理函数目录，包括地址模式的定义及地址处理函数。
Tools：工程配置目录，包括空间划分及Z-Stack相关配置信息。
ZDO：ZDO目录
ZMac：MAC层目录，包括MAC层参数配置及MAC层LIB库函数回调处理函数。
ZMain：主函数目录，包括入口函数及硬件配置文件。
Output：输出文件目录，由IAR IDE自动生成。

前提说明：

此报告文档目的是基于能够使用 `ZigBee` 协议栈进行简单的组网来实现无线数据通信，对协议栈内的具体协议标准原理以及各种软硬件知识、操作系统、通信原理、数据结构、计算机网络、计算机组成原理等没有具体研究. 即能跑起示例工程就算成功...

文档阅读说明：报告文档是基于示例工程 `GenericApp` 来进行学习研究.总体上先对 `Zstack` 的一些需要了解的基本概念和语术进行解释，然后对协议栈运行核心 `OSAL` 进行研究学习，最后通过实际工程示例 `GenericApp` 来说明 `Zstack` 的大致运行流程...

基本概念

- 协议：是一系列的通信标准，通信双方需要按照这一标准进行正常的数据发送和接收
- 协议栈：是协议的具体实现形式

协议栈是协议的实现，可以理解为代码，库函数，供上层应用调用，协议较底下的层与应用是相互独立的。商业化的协议栈只提供你接口。每个厂家的协议栈是有区别的，比如TI的`BLE`协议栈和`nordic`的`BLE`协议栈就有很大的不同

如何使用ZigBee协议栈

以简单的无线数据通信为例，其一般步骤为：

① 组网：调用协议栈组网函数、加入网络函数，实现网络的建立和节点的加入

② 发送：发送节点调用协议栈的发送函数，实现数据无线发送

③ 接收：接收节点调用协议栈的无线接收函数，实现无线数据接收

协调器、路由和终端

设备类型	作用描述
协调器	**启动** 和 **控制** 网络，**协调器** 存储关于 **网络** 信息，包括作为 **认证中心**和 **存储安全密钥**
路由器	扩展网络覆盖面，在障碍周围 **动态路由** ，并且提供 **备份路由** 以防止网络拥挤和设备失败，**路由器** 可以联系到其它路由器，并且支持子设备
终端设备	**终端设备** 可以发送或接收一个信息，但是不能执行任何路由操作，**终端** 必须被联系到 **协调器** 或者 **路由器** ，不支持子设备

参考： zigbee协调器作用

协调器： （coordinator）

每个zigbee网络只允许有一个zigbee的协调器，协调器首先选择一个信道和网络标识(PAN ID)，然后开始这个网络.因为协调器是整个网络的开始，他具有网络的最高权限，是整个网络的维护者，还可以保持间接寻址用的表格绑定，同时还可以设计安全中心和执行其他动作，保持网络其他设备的通信。

路由器： （router）

路由器是一种支持关联的设备，能够实现其他节点的消息转发功能。Zigbee的树形网络可以有多个zigbee路由器设备，zigbee的星型网络不支持zigbee的路由器设备。

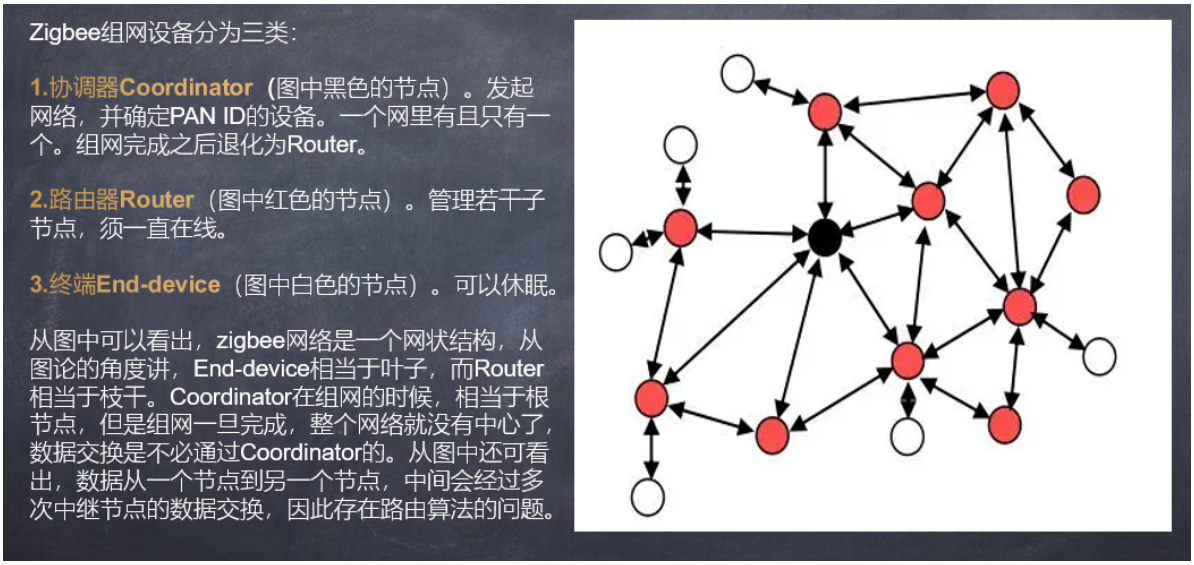
终端： （end）

Zigbee终端节点是具体执行的数据采集传输的设备，他不能转发其他节点的消息。

在z-stack v1.4.1,设备类型的决定通常在编译时间通过编译选项(ZDO_COORDINATOR和RTR_NWK)来确定的.所有的实例应用被提供用分开的项目文件去编译每一个设备类型.

尽可能的建立一个映像既有Coordinator又有Router功能.在运行时选择设备类型.看SOFT_START编译选项

参考：Zigbee的协调器、路由器、终端的概念



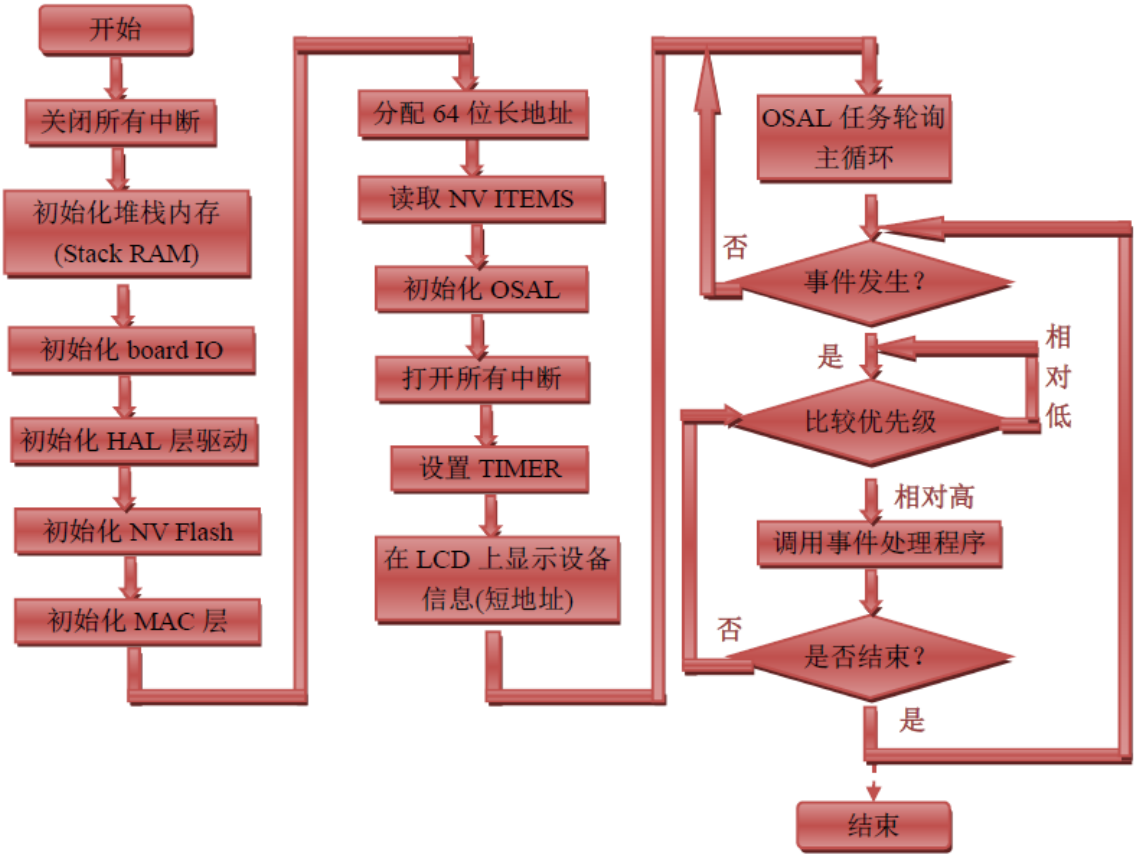
OSAL

Z-stack 协议栈依靠协议栈内部的 OS (`OSAL`) 才能运行起来， `OSAL` 提供以下服务和管理： `信息同步` 、 `任务同步` 、 `时间管理` 、 `中断管理` 、 `任务管理` 、 `内存管理` 、 `电源管理` 以及 `非易失存储管理` ，它并不是 `ZigBee` 协议中的具体层，只是模拟一个操作系统

主要提供以下功能：

- 任务注册、初始化和启动
- 任务间的同步、互斥信息交换
- 中断处理、时间管理
- 存储器分配和管理

执行流程



所有函数说明

> 对项目的入口文件Zmain.c 的 main 内可能会执行到的各函数进行讲解

`osal_int_disable( INTS_ALL );` 关闭总中断

`HAL_BOARD_INIT();` 初始化硬件设备，对晶振、系统时钟的设置测试以及 LED 和各引脚等设置

`zmain_vdd_check();` 确保电源电压足够高以运行

`InitBoard( OB_COLD );` 初始化检测外围 `IO` 设备是否正常

`osal_nv_init( NULL );` 初始化 `NV Flase` 掉电不丢失存储器

`ZMacInit();` 初始化 `MAC` 层

`zmain_ext_addr();` 确定扩展地址

```
#if defined ZCL_KEY_ESTABLISH
  // Initialize the Certicom certificate information.
  zmain_cert_init();
#endif

  // Initialize basic NV items
  zgInit();

#ifndef NONWK
  // Since the AF isn't a task, call it's initialization routine
  afInit();
#endif
```

`osal_init_system();` `osal` 初始化函数

`osal_int_enable( INTS_ALL );` 开启总中断

`zmain_dev_info();` 驱动信息的显示

`zmain_lcd_init();` 驱动信息的显示

`WatchDogEnable( WDTIMX );` 看门狗使能

`osal_start_system();` **系统正式启动运行**

重点关注函数

`osal_init_system();`

```
uint8 osal_init_system( void )
{
  // Initialize the Memory Allocation System
  osal_mem_init();//初始化内存分配系统
  // Initialize the message queue
  osal_qHead = NULL;//初始化消息队列
  // Initialize the timers
  osalTimerInit(); //初始化计时器变量
  // Initialize the Power Management System
```

```
  osal_pwrmgr_init();//初始化电源管理系统
  // Initialize the system tasks.
  osalInitTasks();//初始化系统任务
  // Setup efficient search for the first free block of heap.
  osal_mem_kick();//堆的空闲块的高效搜索
  return ( SUCCESS );
}
```

`osalInitTasks()` 为该函数的重点关注函数，初始化系统任务

该函数的定义在用户操作的 **应用层** 中， 在`OASL_GenericAPP.c` 中定义，在 `OSAL_Tasks.c` 中声明

(即用户需要在应用层中定义一个 `OSAL_XXXX.c` 文件根据需要修改定义自己的任务)

此函数为每个任务调用初始化函数，设置了任务的优先级

```
void osalInitTasks( void )
{
  uint8 taskID = 0;

  tasksEvents = (uint16 *)osal_mem_alloc( sizeof( uint16 ) * tasksCnt);
  osal_memset( tasksEvents, 0, (sizeof( uint16 ) * tasksCnt));

  macTaskInit( taskID++ );
  nwk_init( taskID++ );
  Hal_Init( taskID++ );
#if defined( MT_TASK )
  MT_TaskInit( taskID++ );
#endif
  APS_Init( taskID++ );
#if defined ( ZIGBEE_FRAGMENTATION )
  APSF_Init( taskID++ );
#endif
  ZDApp_Init( taskID++ );
#if defined ( ZIGBEE_FREQ_AGILITY ) || defined ( ZIGBEE_PANID_CONFLICT )
  ZDNwkMgr_Init( taskID++ );
#endif
  GenericApp_Init( taskID );//用户创建的任务
}
```

`GenericApp_Init( taskID )` 为**通用程序任务初始化函数**

此任务是根据想用户想要完成的自定义任务需要**修改的地方**，其它地方为官方提供的配置基本不需要修改

该函数的定义在用户操作的 **应用层** 中， `GenericApp.c` 文件下

该文件下还有 `const pTaskEventHandlerFn tasksArr[]` 任务索引地址

这个数组中元素顺序和上面的任务初始化调用相同

```
const pTaskEventHandlerFn tasksArr[] = {
  macEventLoop,
  nwk_event_loop,
  Hal_ProcessEvent,
#if defined( MT_TASK )
```

```
  MT_ProcessEvent,
#endif
  APS_event_loop,
#if defined ( ZIGBEE_FRAGMENTATION )
  APSF_ProcessEvent,
#endif
  ZDApp_event_loop,
#if defined ( ZIGBEE_FREQ_AGILITY ) || defined ( ZIGBEE_PANID_CONFLICT )
  ZDNwkMgr_event_loop,
#endif
  GenericApp_ProcessEvent
};
```

`SampleApp_ProcessEvent()` 为通用应用程序时间处理器 . 此函数调用以处理任务的所有事件，包括计时器、消息和任何用户自定的事件

定义上，我们把系统需要执行的事情叫做事件，作为用户，我们面对的大多是 `application` 层，我们希望添加自己的事件，比如自己写一个函数让 `osal` 去做，这是个事件， `osal` 是一个死循环的系统，`cpu` 一直检查是否有事件执行. 然后是任务，多个事件可以是一个任务 . 像是一个快递员，一栋楼是一个任务，有多个快递在这栋楼里. 而有多少栋这样的楼，就在 `osalInitTasks` 函数中初始化的各项任务中. 对我们来说，其中最重要的就是`用户的自定义任务` ， 而事件的定义的索引在 `tasksArr[]` 数组中.

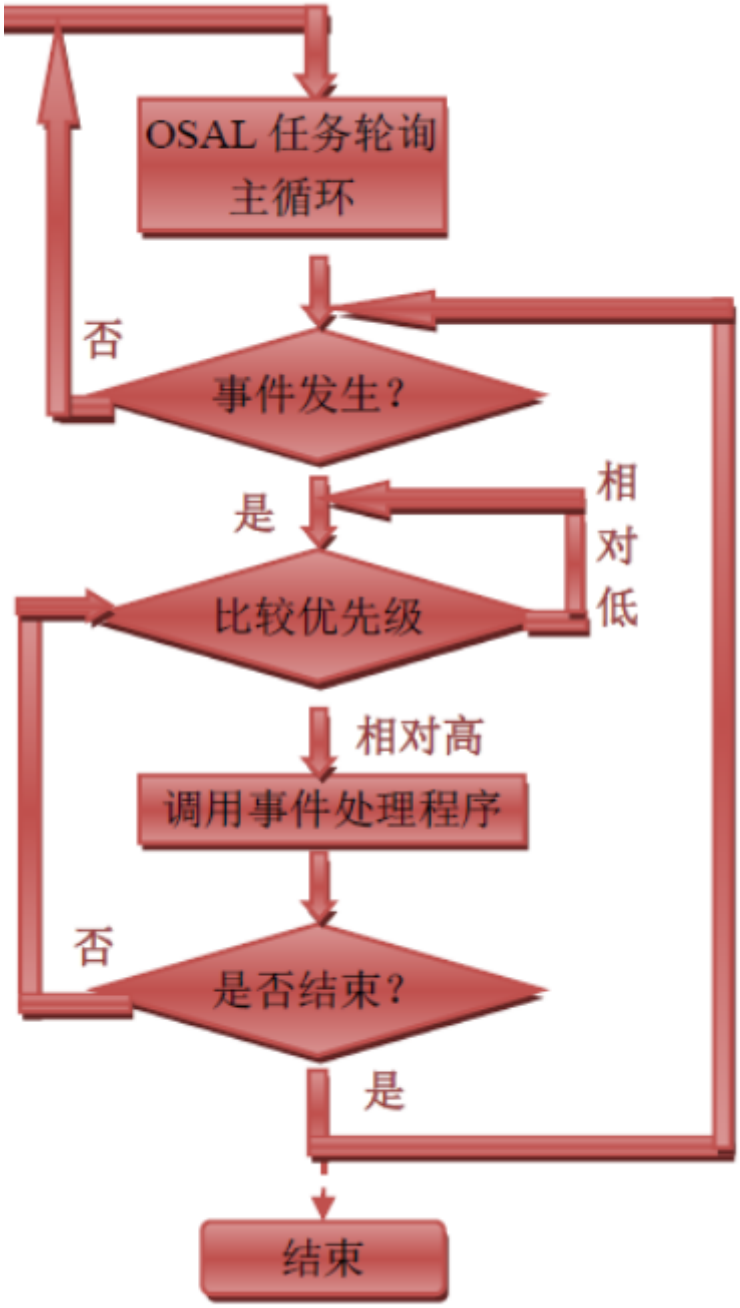
完成了 `OSAL` 的初始化，接下来看 `OSAL` 的具体运行

`osal_start_system();`

```
void osal_start_system( void )
{
#if !defined ( ZBIT ) && !defined ( UBIT )
  for(;;)  // Forever Loop
#endif
  {
    osal_run_system();
  }
}
```

`ZBIT` 和 `UBIT` 是用于**TI内部测试**所使用的一个宏定义，默认下是无定义的。

死循环执行 `osal_run_system()` 函数




```
void osal_run_system( void )
{
  uint8 idx = 0;//任务索引

  osalTimeUpdate();//轮询 Timer 和 Uart
  Hal_ProcessPoll();
  //从优先级别最高的事件开始遍历，找到当前优先级最高的未处理事件
  do {
    if (tasksEvents[idx])  // Task is highest priority that is ready.
    {
      break;
    }
  } while (++idx < tasksCnt);

  if (idx < tasksCnt)//执行该事件
  {
    uint16 events;
    halIntState_t intState;
```

```
    HAL_ENTER_CRITICAL_SECTION(intState);//进入临界区
    events = tasksEvents[idx];//取出本次待处理任务的事件
    tasksEvents[idx] = 0;  // Clear the Events for this task.清除本次任务事件
    HAL_EXIT_CRITICAL_SECTION(intState);//退出临界区

    activeTaskID = idx;
    events = (tasksArr[idx])( idx, events );//通过指针调用事件执行函数！！！
    activeTaskID = TASK_NO_TASK;//执行成功给任务ID 标志位
    //进入临界区保存未处理的事件
    HAL_ENTER_CRITICAL_SECTION(intState);
    tasksEvents[idx] |= events;  // Add back unprocessed events to the current
task.
    HAL_EXIT_CRITICAL_SECTION(intState);
  }
//如果定义了低功耗模式，在没有任务的情况下会进入休眠
#if defined( POWER_SAVING )
  else  // Complete pass through all task events with no activity?
  {
    osal_pwrmgr_powerconserve();  // Put the processor/system into sleep
  }
#endif

  /* Yield in case cooperative scheduling is being used. */
#if defined (configUSE_PREEMPTION) && (configUSE_PREEMPTION == 0)
  {
    osal_task_yield();
  }
#endif
}
```

`osal_run_system()` 函数的主要执行的步骤

1. 扫描哪个事件触发了，置相应标志位
2. 轮询 `Timer` 和 `Uart`
3. 从优先级最高的事件开始执行，找到当前优先级最高的未处理事件执行
4. 如果没有任务执行就进入休眠状态

通过调用 `taskArr[idx]` 会调用到 `GenericApp_ProcessEvent` 函数进行具体的用户自定义事件处理

`GenericApp` 示例的应用解析

这个实验是两个模块互相绑定后可以对传数据，模块绑定之后，两个模块之间互相传递字符串 `helloworld`

实验现象说明： 首先启动一个网络协调器，协调器如果建立网络成功后，会在 `LCD` 上显示该节点为协调者同时显示网络 `ID` 号 . 然后打开一个终端节点或路由器的电源，此时节点会自动加入网络. 加入网络成功后，节点会显示自己的节点类型、网络地址和父节点的网络地址

节点加入网络后，首先把主机模块的摇杆往右拨一下，然后把要绑定模块的摇杆也往右拨一下，如果两边的 `LED4` 都熄灭或是点亮后马上熄灭，表示绑定成功. 绑定成功后，两个节点就开始相互定时发送数据，并在对方的 `LCD` 屏显示出来，发送的数据为 `Hello Wrold` 此时如果把相互作绑定模块中的一个摇杆往左拨以下，可以发送 `Match Description Request` 命令，对方则显示 `Match Description Request` 信息

待完善...

小组的后续目标

- [ ] 继续学习 `zstack` 协议栈，了解程序的大致运行流程
- [ ] 结合课程单片机去修改配置文件，能够跑起示例工程